

# HDOC DAY-33 ASSIGNMENT

## Associative Property of Matrix Addition and Multiplication

### 1. Problem Understanding

The task is to write a menu-driven C program using separate functions to verify the associative property of matrix addition and matrix multiplication using variable-length 2D integer arrays.

Menu options:

1. Read three matrices A, B and C of the same dimensions (m x n) and verify  $(A + B) + C = A + (B + C)$ .
2. Read matrices A (m x n), B (n x p) and C (p x q) and verify  $(A \times B) \times C = A \times (B \times C)$ .
3. Exit.

For each operation, the matrices obtained on both sides of the equation are displayed and the program states whether the associative property is verified.

### 2. Standard Test Cases

#### A. Matrix Addition

| Test Case | Choice | m | n | Matrix A      | Matrix B      | Matrix C      | Expected Result                    |
|-----------|--------|---|---|---------------|---------------|---------------|------------------------------------|
| TC1       | 1      | 2 | 2 | 1 2 / 3 4     | 5 6 / 7 8     | 1 1 / 1 1     | Both sides = 7 9 / 11 13; Verified |
| TC2       | 1      | 2 | 3 | 1 0 2 / 3 1 4 | 2 1 0 / 1 2 3 | 4 2 1 / 0 1 2 | Both sides equal; Verified         |

#### B. Matrix Multiplication

| Test Case | Choice | m | n | p | q | Matrix A  | Matrix B      | Matrix C  | Expected Result                    |
|-----------|--------|---|---|---|---|-----------|---------------|-----------|------------------------------------|
| TC3       | 2      | 2 | 2 | 2 | 2 | 1 2 / 3 4 | 2 0 / 1 2     | 1 1 / 0 1 | Both sides = 4 8 / 10 18; Verified |
| TC4       | 2      | 1 | 2 | 3 | 1 | 1 2       | 1 0 2 / 0 1 3 | 2 / 1 / 1 | Both sides = 11; Verified          |

### 3. Algorithm

#### A. Common Menu Logic

1. Start.
2. Display the menu.
3. Read the user choice.
4. Use switch-case to call the required function.
5. If choice is 3, exit the program.
6. If the choice is invalid, display "Invalid choice".
7. Stop.

#### B. Verify Associative Property of Matrix Addition

1. Read m and n.
2. Declare matrices A, B, C and result matrices of size m x n.

3. Read all elements of A, B and C.
4. Calculate left side as  $(A + B) + C$ .
5. Calculate right side as  $A + (B + C)$ .
6. Display the left-side matrix.
7. Display the right-side matrix.
8. Compare corresponding elements of both result matrices.
9. If all elements are equal, display that matrix addition is associative; otherwise display that it is not verified.

### **C. Verify Associative Property of Matrix Multiplication**

1. Read m, n, p and q.
2. Declare A of size  $m \times n$ , B of size  $n \times p$  and C of size  $p \times q$ .
3. Read all elements of A, B and C.
4. Calculate  $AB = A \times B$ , then calculate left side =  $AB \times C$ .
5. Calculate  $BC = B \times C$ , then calculate right side =  $A \times BC$ .
6. Display the left-side matrix.
7. Display the right-side matrix.
8. Compare corresponding elements of both result matrices.
9. If all elements are equal, display that matrix multiplication is associative; otherwise display that it is not verified.

## 4. C Program

```
#include <stdio.h>
void addition()
{
    int m, n, i, j;
    printf("Enter rows and columns: ");
    scanf("%d%d", &m, &n);
    int A[m][n], B[m][n], C[m][n];
    int left[m][n], right[m][n];
    int flag = 1;
    printf("Enter matrix A:\n");
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &A[i][j]);
        }
    }
    printf("Enter matrix B:\n");
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &B[i][j]);
        }
    }
    printf("Enter matrix C:\n");
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &C[i][j]);
        }
    }
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
        {
            left[i][j] = (A[i][j] + B[i][j]) + C[i][j];
            right[i][j] = A[i][j] + (B[i][j] + C[i][j]);
        }
    }
    printf("(A+B)+C:\n");
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
        {
            printf("%d ", left[i][j]);
        }
        printf("\n");
    }
    printf("A+(B+C):\n");
```

```

for(i = 0; i < m; i++)
{
    for(j = 0; j < n; j++)
    {
        printf("%d ", right[i][j]);
        if(left[i][j] != right[i][j])
        {
            flag = 0;
        }
    }
    printf("\n");
}
if(flag == 1)
{
    printf("Matrix addition is associative.\n");
}
else
{
    printf("Associative property is not verified.\n");
}
}

void multiplication()
{
    int m, n, p, q, i, j, k;
    printf("Enter m, n, p and q: ");
    scanf("%d%d%d%d", &m, &n, &p, &q);
    int A[m][n], B[n][p], C[p][q];
    int AB[m][p], BC[n][q];
    int left[m][q], right[m][q];
    int flag = 1;
    printf("Enter matrix A:\n");
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &A[i][j]);
        }
    }
    printf("Enter matrix B:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < p; j++)
        {
            scanf("%d", &B[i][j]);
        }
    }
    printf("Enter matrix C:\n");
    for(i = 0; i < p; i++)
    {
        for(j = 0; j < q; j++)
        {
            scanf("%d", &C[i][j]);
        }
    }
}

```

```

}
for(i = 0; i < m; i++)
{
    for(j = 0; j < p; j++)
    {
        AB[i][j] = 0;
        for(k = 0; k < n; k++)
        {
            AB[i][j] = AB[i][j] + A[i][k] * B[k][j];
        }
    }
}
for(i = 0; i < m; i++)
{
    for(j = 0; j < q; j++)
    {
        left[i][j] = 0;
        for(k = 0; k < p; k++)
        {
            left[i][j] = left[i][j] + AB[i][k] * C[k][j];
        }
    }
}
for(i = 0; i < n; i++)
{
    for(j = 0; j < q; j++)
    {
        BC[i][j] = 0;
        for(k = 0; k < p; k++)
        {
            BC[i][j] = BC[i][j] + B[i][k] * C[k][j];
        }
    }
}
for(i = 0; i < m; i++)
{
    for(j = 0; j < q; j++)
    {
        right[i][j] = 0;
        for(k = 0; k < n; k++)
        {
            right[i][j] = right[i][j] + A[i][k] * BC[k][j];
        }
    }
}
printf("(A*B)*C:\n");
for(i = 0; i < m; i++)
{
    for(j = 0; j < q; j++)
    {
        printf("%d ", left[i][j]);
    }
    printf("\n");
}

```

```

}
printf("A*(B*C):\n");
for(i = 0; i < m; i++)
{
    for(j = 0; j < q; j++)
    {
        printf("%d ", right[i][j]);
        if(left[i][j] != right[i][j])
        {
            flag = 0;
        }
    }
    printf("\n");
}
if(flag == 1)
{
    printf("Matrix multiplication is associative.\n");
}
else
{
    printf("Associative property is not verified.\n");
}
}
int main()
{
    int choice;

    do
    {
        printf("\nMENU\n");
        printf("1. Verify associative property of matrix addition\n");
        printf("2. Verify associative property of matrix multiplication\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                addition();
                break;
            case 2:
                multiplication();
                break;
            case 3:
                printf("Program ended.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    }
    while(choice != 3);
    return 0;
}

```

}

## 5. Compilation and Execution Evidence

```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)~/Desktop
$ gedit question.c

** (gedit:21794): WARNING **: 13:25:30.154: Could not load Peas repository: Typelib file for
namespace 'Peas', version '1.0' not found

** (gedit:21794): WARNING **: 13:25:30.155: Could not load PeasGtk repository: Typelib file
for namespace 'PeasGtk', version '1.0' not found

(kali@kali)~/Desktop
$ gcc question.c -o question

(kali@kali)~/Desktop
$ ./question

MENU
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 1
Enter rows and columns: 2
2
Enter matrix A:
1 2 3 4
Enter matrix B:
5 6 7 8
Enter matrix C:
1 1 1 1
(A+B)+C:
7 9
11 13
A+(B+C):
7 9
11 13
Matrix addition is associative.

MENU
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help

Enter your choice: 1
Enter rows and columns: 2 3
Enter matrix A:
1 0 2 3 1 4
Enter matrix B:
2 1 0 1 2 3
Enter matrix C:
4 2 1 0 1 2
(A+B)+C:
7 3 3
4 4 9
A+(B+C):
7 3 3
4 4 9
Matrix addition is associative.

MENU
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 2
Enter m, n, p and q: 2 2 2 2
Enter matrix A:
1 2 3 4
Enter matrix B:
2 0 1 2
Enter matrix C:
1 1 0 1
(A*B)+C:
4 8
10 18
A*(B+C):
4 8
10 18
Matrix multiplication is associative.

MENU
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 2
```

```
Session Actions Edit View Help
Enter matrix A:
1 2 3 4
Enter matrix B:
2 0 1 2
Enter matrix C:
1 1 0 1
(A*B)+C:
4 8
10 18
A*(B+C):
4 8
10 18
Matrix multiplication is associative.

MENU
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 2
Enter m, n, p and q: 1 2 3 1
Enter matrix A:
1 2
Enter matrix B:
1 0 2 0 1 3
Enter matrix C:
2 1 1
(A*B)*C:
12
A*(B+C):
12
Matrix multiplication is associative.

MENU
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 3
Program ended.

(kali@kali)-[~/Desktop]
```

6. Verification of All Test Cases

| Test Case | Operation      | Dimensions       | Inputs                                      | Expected Result            | Actual Result              | Status |
|-----------|----------------|------------------|---------------------------------------------|----------------------------|----------------------------|--------|
| TC1       | Addition       | m=2, n=2         | A=1 2/3 4; B=5 6/7 8; C=1 1/1 1             | 7 9 / 11 13; Verified      | 7 9 / 11 13; Verified      | Pass   |
| TC2       | Addition       | m=2, n=3         | A=1 0 2/3 1 4; B=2 1 0/1 2 3; C=4 2 1/0 1 2 | Both sides equal; Verified | Both sides equal; Verified | Pass   |
| TC3       | Multiplication | m=2,n=2,p=2, q=2 | A=1 2/3 4; B=2 0/1 2; C=1 1/0 1             | 4 8 / 10 18; Verified      | 4 8 / 10 18; Verified      | Pass   |
| TC4       | Multiplication | m=1,n=2,p=3, q=1 | A=1 2; B=1 0 2/0 1 3; C=2/1/1               | 11; Verified               | 11; Verified               | Pass   |